

ALAPFOGALMAK

- **algoritmusnak** nevezünk minden olyan jól definiált számítási eljárást, amely bemeneti értékekből kimeneti értékeket állít elő
 - a **helyesség** azt jelenti, hogy ha az algoritmus ad kimeneti értéket akkor az helyes lesz
 - a **teljeség** azt jelenti, hogy minden inputra helyes kimeneti értéket ad
- **adatszerkezet** az adatok tárolására és szervezésére szolgáló módszer, amely lehetővé teszi a hatékony hozzáférést és módosítást is

Algoritmusok hatékonysága

- az algoritmusok hatékonyságát aszimptotikus jelölésekkel mérjük
- futási idő mérés
 - mennyi művelet fut le az algoritmus során, mennyi idő alatt áll meg (lépések száma)
 - elemi műveletek: +, -, x = 5, if, print
 - összetett műveletek: for
 - feltesszük, hogy egy sor végrehajtása konstans időnek felel meg
 - $T(n)$ = összfutási idő, n = bemenet mérete (konstans)
 - vizsgálat során feltesszük, hogy n tart a végtelenbe, $T(n)$ pedig monoton nő
 - ezért elhagyhatjuk a legnagyobb kitevőjű tagon kívül a többit belőle (felsőkorlátos aszimptotikusság, azaz ordó végett)
 - pl $T(n) = 15n^2 + 20n + 10$ esetén csak a $15n^2$ -t vesszük
- **Ordó ($O(g(n))$) - Aszimptotikus felső korlát**
 - általában a legrosszabb esetet jelenti
 - azt jelenti, hogy az algoritmus futási ideje legfeljebb milyen gyorsan nő a bemenet (n) függvényében
 - formálisan, létezik c és n_0 pozitív konstans úgy, hogy megfelelően nagy n esetén ($n \geq n_0$), akkor $0 \leq f(n) \leq c \cdot g(n)$
 - $f(n)$ felülről korlátozza a $c \cdot g(n)$ -t
- **Théta ($\Theta(g(n))$) - Aszimptotikus Éles korlát, közre fogva**
 - azt jelenti, hogy a függvény alulról és felülről is korlátozva van $g(n)$ konstans szorosaival, tehát az algoritmus nagyságrendileg pontosan úgy skálázódik, mint $g(n)$.

Theta:

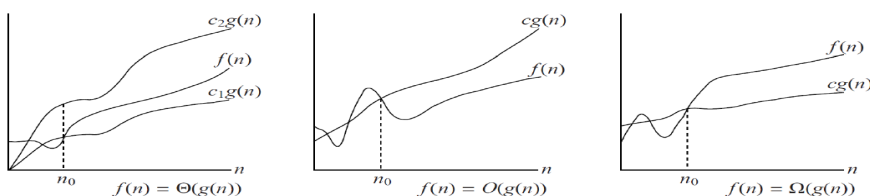
$\Theta(g(n)) = \{f(n) : \text{létezik } c_1, c_2 \text{ és } n_0 \text{ pozitív állandó úgy, hogy } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ teljesül minden } n \geq n_0 \text{ esetén}\}.$

Ordó:

$O(g(n)) = \{f(n) : \text{létezik } c \text{ és } n_0 \text{ pozitív állandó úgy, hogy } 0 \leq f(n) \leq c g(n) \text{ teljesül minden } n \geq n_0 \text{ esetén}\}.$

Omega:

$\Omega(g(n)) = \{f(n) : \text{létezik } c \text{ és } n_0 \text{ pozitív konstans úgy, hogy } 0 \leq c g(n) \leq f(n) \text{ teljesül minden } n \geq n_0 \text{ esetén}\}.$



RÉSZPROBLÉMÁRA BONTÓ ALGORITMUSOK

- a bonyolult feladatok megoldásának egyik leghatékonyabb módja
- három fő stratégiát különböztetünk meg: **oszd-meg-és-uralkodj**, **dinamikus programozás** és **mohó algoritmus**

1. Oszd-meg-és-uralkodj (Divide and Conquer)

- ez a módszer a problémát több diszjunkt (egymást nem átfedő), az eredetihez hasonló (homogén), de kisebb méretű részfeladatra bontja és ezeket rekurzívan megoldja, majd a részeredményeket összevonja (kombinálja) a végleges megoldássá
- lépései:
 - **Felosztás:** a probléma darabolása (top-down, a nagy feladatot bontsa szét)
 - **Uralkodás**
 - a részfeladatok rekurzív megoldása (vagy közvetlen megoldás, ha a méret triviális - alapeset)
 - nem minden oszd meg és uralkodj algoritmus használ rekurziót és nem is minden rekurzió oszd meg és uralkodj típusú
 - **rekurzió** azaz, hogy önmagát ismétli az algoritmus, tehát saját maga által definiált műveletet hajt végre
 - **alapeset:** megoldást előre tudjuk, könnyű kiszámítani
 - **összetett eset:** nem tudjuk előre a megoldást, a kiszámításához felhasználjuk a kisebb feladatok (alapesetektől kiindulva) megoldásait
 - végtelen rekurzió
 - kölcsönös rekurzió
 - a függvény, b függvényt és b függvény, a függvényt hívja
 - jó esetben van megállási feltétel
 - **Összevonás:** a részmegoldások egyesítés
 - futásidejének elemzésére a **rekurziós fák** vagy a **Mester-tétel** használható, esetleg a **helyettesítő módszer**
 - fel kell oldalni a rekurziót, mivel általában rekurzív összefüggésekkel írható fel (pl. $T(n) = 2T(n/2) + n$), viszont ránézésre ezekből nem tudjuk megmondani, hogy az algoritmus hatékony-e ezért kell felbontani
 - **Mester-tétel** a $T(n) = a \cdot T(n/b) + f(n)$ alakú összefüggésekre ad megoldást
 - a = részproblémák száma
 - $T(n/b)$ = minden részprobléma b -ed részére osztja az előző problémát és ez annak az időigény
 - $f(n)$ = időigénye a részproblémák megoldásainak összefűzéséhez
 - **helyettesítő módszer**, akkor ha nincs kész sablon vagy ha a rekurziós fa alapján a sejtésünket be akarjuk bizonyítani
 - megsejtjük a megoldás formáját (pl. azt tippeljük, hogy $T(n) = O(n \log n)$) és matematikai indukcióval bizonyítjuk, és meghatározzuk közben a definícióban szereplő konstansokat (c, n_0)
 - $T(n) \leq c \cdot f(n)$
 - példák: Összefésülő rendezés (Merge Sort), Gyorsrendezés (Quick Sort), Bináris keresés

2. Dinamikus Programozás (DP)

- itt is részfeladatokra bontás történik, de vannak egyéb feltételek:
 - a részfeladatok nem függetlenek, hanem **átfedőek** (átfedő részproblémák)
 - a probléma **optimális részstruktúrával** rendelkezik
 - az eredeti probléma egy optimauma (optimális megoldása) a részfeladatok optimumából épül fel
- 2 megoldási módja van
 - **táblázatkitöltés** (tabulation) - **bottom-up építkezés**
 - iteratív megvalósítás, a kiszámolt részproblémák értékeit egy tömbbe elmentjük
 - **memoization** - **top-down építkezés**
 - rekurziót használ, a már kiszámolt értékeket kulcs-érték formájában tároljuk
 - akkor használjuk ha nem kell mindent kiszámolni, teljes megoldásból visszafejtéssel megtudunk szerkeszteni egy optimális megoldást
- példa (tipikusan optimalizálási feladatoknál érdemes használni)
 - hátizsák probléma (egészes eset csak, töredékesnél mohó)
 - létrehozunk egy kétdimenziós táblázatot (mátrixot), ahol a sorok a tárgyakat, az oszlopok a hátizsák aktuális kapacitását jelentik (0-tól S -ig)
 - a táblázatot sorról sorra töltjük fel a korábbi részeredmények alapján: minden cellánál eldöntjük, hogy az adott tárgyat berakjuk-e (ha befér) vagy kihagyjuk, attól függően, melyik ad nagyobb értéket
 - pénzváltási probléma (minimális érmeszám meghatározása), ahol a célösszeget felépíthetjük kisebb összegek optimális megoldásaiból
 - pl: vannak 1-es, 5-ös, 10-es érmék akkor hogy bontsuk fel 14-et, stb
 - a DP algoritmus sorban kiszámolja a minimum értéket minden részösszegre, pl: 14 esetén: 1-re, 2-re, ... 14-re és a korábbi eredményeket újra felhasználja: $dp[i] = \min(dp[i], dp[i - \text{érme}] + 1)$
 - $dp[0] = 0$
 - $i = 1$ -től 4 -ig: mivel csak az 1-es érme kisebb vagy egyenlő náluk, a tömb simán növekszik: $dp[1] = 1$, $dp[2] = 2$, $dp[3] = 3$, $dp[4] = 4$
 - $i = 5$ -től már van ötös érme is:
 - 1-es érmével: $dp[5-1] + 1 = 5$ érme kell
 - 5-ös érmével: $dp[5-5] + 1 = 1$ érme kell - természetesen ezt választjuk
 - $i = 6$ -tól 9 -ig:
 - $dp[6] = dp[6-5] + 1 = dp[1] + 1 = 2$ (egy 5-ös + egy 1-es)
 - $dp[9] = dp[9-5] + 1 = dp[4] + 1 = 5$ (egy 5-ös + négy 1-es)
 - és így tovább csak még ugye 10-től bejön a 10-es érme is
 - a végső válasz a $dp[14]$ cellában van, ami 5 (egy 10-es és négy darab 1-es)

3. Mohó Algoritmusok (Greedy)

- a mohó stratégia minden lépésben az aktuális (lokális) legjobbnak tűnő választást teszi meg, abban a reményben, hogy ez a globális optimumhoz vezet
 - nem vizsgálja felül a korábbi döntéseit
 - mindig csak a következő lépést látja és ott fogja a legjobb döntést hozni
 - nem lát tovább, pontosabban nem foglalkozik a döntési globális optimummal és a későbbi eseményekre gyakorolt hatásával
- egy lépés egyenlő egy részproblémával
- csak akkor működik és optimális, ha a probléma rendelkezik a mohó választási tulajdonsággal (a lokális optimum egyben része a globálisnak), azaz biztonságos választás, ezzel jön, hogy biztos, hogy az eredeti probléma optimális megoldását kapjuk (optimális részstruktúra)
 - optimális részstruktúra az, hogy a mohó választás után jellemzően, csak egyetlen részfeladat marad, amit meg kell oldani
 - pl ha megvan a teljes út a csúcsra (optimális megoldás), akkor ennek az útnak bármelyik szakasza (pl. a felétől a csúcsig) is a lehető legjobb út kell, hogy legyen azon a szakaszon
- felülről lefelé épít (top-down, pl: a legnagyobb fajlagos súllyal (érték/súly) kezd)
- nagyon gyors, de nem minden problémára ad optimális megoldást
- példa
 - töredékes hátizsák probléma (ahol a tárgyak darabolhatók – a fajlagos érték alapján választunk), Dijkstra algoritmus
 - a 0-1 (egész) hátizsák probléma nem oldható meg mohón, oda DP kell
 - ismétléses variáns esetén minden kombináció megadása és a legjobb kiválasztása, futásiideje: $O(S^n)$
 - S a kapacitás, n a tárgyak száma

Hátizsák feladat összefoglaló

- 2 variáns ismétléses és ismétlés nélküli
 - illetve egész (dp) vagy töredékes (mohó)
- S kapacitású táska, $E1$, $E2$ értékű tárgyak
- brute force (nyers erő) megoldás
 - minden kombináció megadása és a legjobb kiválasztása
 - futásiideje: $O(S^n)$
- dinamikus programozás, részfeladatokra bontás
 - veszünk egy s kapacitást amire kiszámoljuk az optimális megoldást (pl: $S=15$, akkor s lehet pl 10)
 - ha valamelyik i tárgyat kivesszük ebből az optimális megoldásból akkor $(s - S * i)$ -ra kapunk optimális megoldást
 - ilyenkor az a cél, hogy végén, hogy az optimális megoldást a fő problémára megkapjuk akkor csak 1 tárgyat kelljen hozzáadni
 - futásiideje: $O(S * n)$

RENDEZŐ ALGORITMUSOK

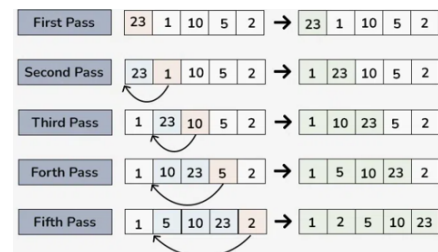
- a rendezés célja egy n elemű input sorozat elemeinek növekvő (vagy csökkenő) sorrendbe állítása (permutáció)
- egy rendezés **stabil**, ha azonos értékű elemek ugyanabban a sorrendbe jelennek meg a kimeneti tömbbe, mint az eredeti bemenetiben, akkor fontos ha vannak kísérő adatok is vagy ha több szempont alapján rendezünk
 - például neveket rendezünk, amikhez tartozik egy életkor is
 - ha a lista már névsorban van (András 20, Béla 20), és kor szerint rendezünk, a stabil rendezés után András továbbra is Béla előtt marad
 - ha nem stabil, akkor Béla András elé kerülhet akár
- **helyben rendezés**
 - helyben rendezés azt jelenti, hogy a bementi tömbön kívül legfeljebb konstans méretű tár ($O(1)$)
 - például a beszűrő rendezés helyben végzett, mert csak egyetlen segédváltozót (kulcs) használ, amibe ideiglenesen kimenti kimenti az éppen mozgatott elemet
 - ezzel szemben az Összefésülő rendezés nem helyben végzett, mert a kész rész összevonásához létre kell hoznia egy új ugyanolyan méretű segéd tömböt a memóriában

ÖSSZEHAISONLÍTÓ RENDEZÉSEK

- ezeknél az elemek sorrendjét kizárólag összehasonlításokkal határozzuk meg. Alsó korlátjuk bizonyítottan $\Omega(n \log n)$
- **beszűrő rendezés, összefésülő rendezés (merge), kupac rendezés, gyors rendezés**

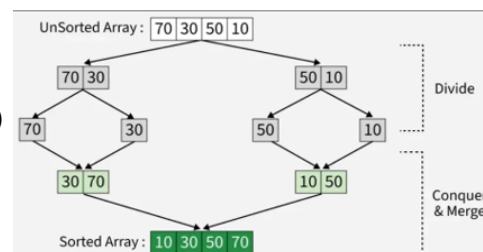
Beszűrő rendezés (insertion sort)

- a kártyalapok rendezéséhez hasonlít
- kicsi és majdnem rendezet tömbök esetén hatékony
- lépései (növekvő sorrendbe rendezésnél)
 - a tömb második elemét hasonlítsuk össze az elsővel, ha a második kisebb akkor cseréljenek helyet
 - majd a 3. elemet hasonlítsuk össze a 2. és 1. elemmel (tehát mindig az előtte lévővel csináljuk) és a megfelelő helyre tesszük
 - ezt addig ismételjük még nem lesz rendezett
- tulajdonságai
 - $O(n^2)$ a legrosszabb esetben a futásideje
 - stabil és helyben rendez



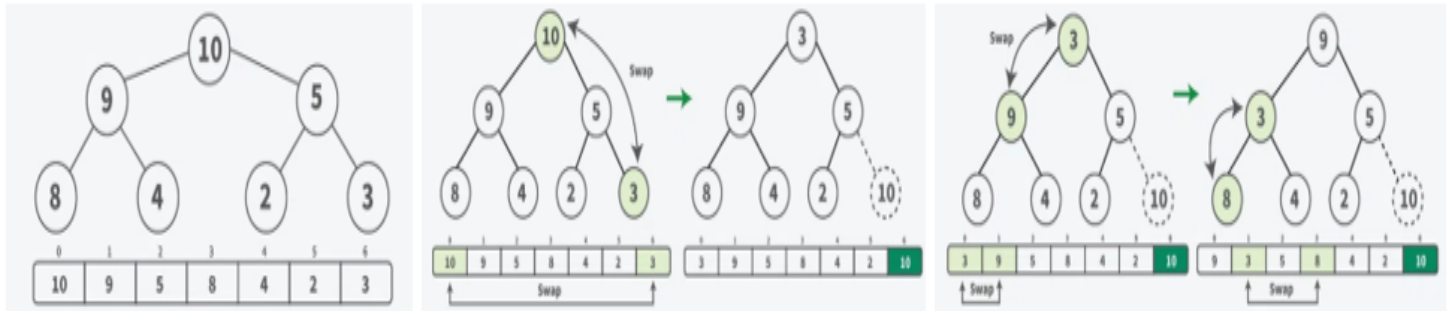
Összefésülő rendezés (merge sort)

- oszd-meg-és-uralkodj elvű
 - a tömböt rekurzívan felezzük, míg 1 elemű nem lesz, majd lentől felfelé összefésüljük őket rendezve (minden kapott tömböt külön rendezzük)
- tulajdonságai
 - $\Theta(n \log n)$ minden esetben a futási idő
 - stabil, de nem helyben rendez (extra tárigénye $O(n)$)



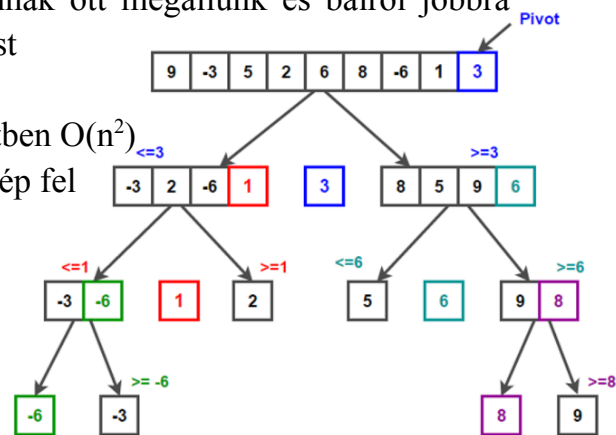
Kupac rendezés (heap sort)

- egy bináris kupac adatszerkezetet épít, ez lehet max-heap, vagy min-heap
 - ez egy majdnem teljes bináris fa, amelyben minden szülő csúcs értéke legalább akkora mint a gyerekeié (max-kupac esetén a gyökérbe a max van)
- a gyökérelém és az utolsó levél (legjobb oldalibb) cseréjével, majd a kupac tulajdonság helyreállításával rendezzi a tömböt, addig még nem lesz jó
- tulajdonságai
 - $O(n \log n)$
 - nem stabil, de helyben rendez



Gyorsrendezés (quick sort)

- oszd-meg-és-uralkodj elvű
- menete
 - pivot (vezér, főelem) elem kiválasztásával indul (pl: utolsó, első vagy középső elem lesz (legrosszabb eset időigényileg), mediánhoz közelibbet, vagy random akár) és a tömböt két részre osztjuk a pivotnál kisebbekre és nagyobbakra és rekurzívan így folytatjuk
 - végül amikor csak 1 elemű tömbök vannak ott megállunk és balról jobbra olvasva a leveleket meg kapjuk a rendezést
- tulajdonságai
 - átlagosan $O(n \log(n))$, de legrosszabb esetben $O(n^2)$
 - utóbbi jó pivot választással ritkán lép fel
 - nem stabil
 - helyben rendez (kis verem használatával)

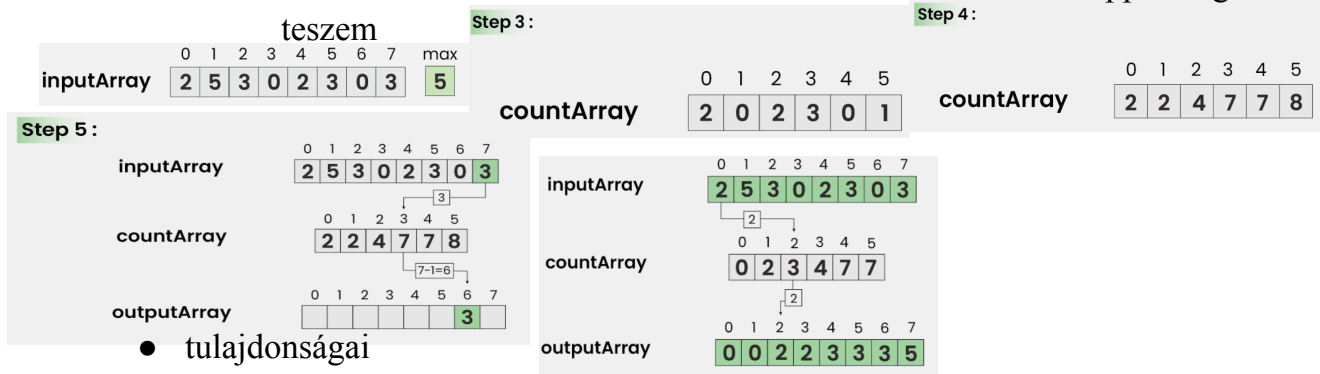


LINEÁRIS IDEJŰ RENDEZÉSEK

- nem összehasonlításra alapulnak, hanem az adatok speciális tulajdonságait (pl. egész számok korlátos tartományában) használják ki
- $O(n)$ futásiidő
- leszámoló rendezés, radix rendezés és edényrendezés

Leszámláló rendezés (counting sort)

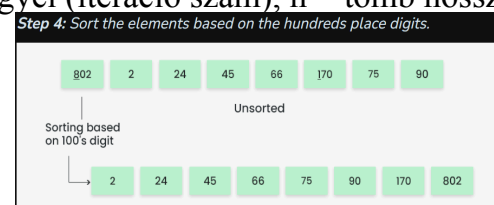
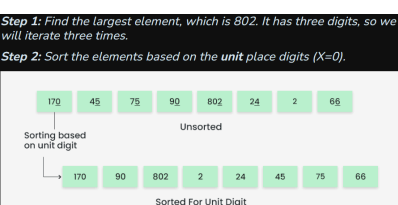
- kulcs alapján rendez (nem összehasonlító rendezés)
- ismerjük a lehetséges értéktartományt (n bemeneti elem mindegyike 0 és egy k közötti egész szám)
- megszámolja miből hány darab van, majd kiszámolja a pozíciókat
 - segéd tömbbe tárolja miből mennyi van (a kulcsok a segéd tömbbe az értéktartomány összes eleme, ami nincs a sorozatba oda 0-át veszünk)
 - a segéd tömb retesz értékét úgy módosítjuk, hogy az előtte lévő retesz értékét hozzáadjuk (alsó indextől indulunk)
 - így ez pozíciókat tárol (pl, hogy abból a számból x darab lehet ami vagy egyenlő azzal vagy kisebb)
 - új tömböt hozok létre az outputnak
 - majd végig megyünk az eredeti sorozaton jobbról balra (azért így, hogy stabil maradjon) és az aktuális elemmel egyező kulcsú (indexű) elemet a segéd táblában frissítem (ha nem 0 akkor kivonok belőle 1-et) és az output tömbbe a frissített tömb indexére az eredeti sorozatból épp vizsgált értéket



- tulajdonságai
 - idő- és tárigénye $O(n + k)$
 - k az értékkészlet mérete, ha nem nagyobb a k , mint az n akkor lineáris ($O(n)$)
 - stabil

Számjegyes rendezés (radix sort)

- helyiértékenként rendez (legkisebbtől kezdve)
 - pl. utolsó számjegy vagy karaktertől kezdve, esetleg dátum esetén a nap -> hónap -> év
 - azonosság alapján az kerül előrébb ami az eredeti tömbbe előrébb van (segédmátrixban vezethető)
 - eredeti tömb mindig felülíródik egy adott karakter / számjegy ellenőrzése után, az alapján, hogyan rendeztük
- tulajdonságok
 - stabil segédrendezést használ (leszámlolót), de nem helyben rendez ($O(n + k)$)
 - stabil rendezés fontos, hogy pl ha egyesek alapján már rendeztük ne boruljon fel a sorrend
 - futásideje $O(d * (n + k))$
 - d = a maximum elemnek a számjegyei (iteráció szám), n = tömb hossz, k = a radix, decimálisoknál 10



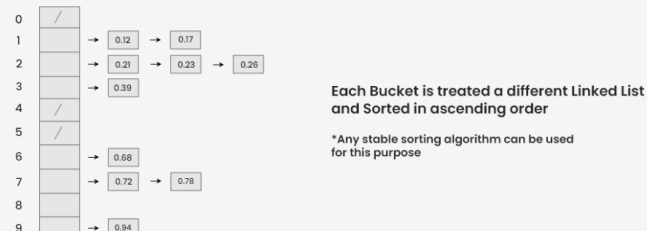
Edényrendezés (bucket sort)

- az elemeket intervallumokba (edényekbe) osztja, minden szám a megfelelő edénybe kerül és azokon belül történik a rendezés (pl: beszűrő rendezéssel, mert stabil rendezés kell), majd összefűzésre kerülnek
 - pl. 10 elem esetén legyen 10 edény, ha mondjuk 0, 1 között vannak a valós számok akkor beszorozzuk az értéket 10-el és lekerekítjük és az adott indexű edénybe rakjuk, utána rendezzük az edények tartalmát, majd összefűzzük őket lentől fel (0. indextől 9.-ig)
- tulajdonságok
 - futásideje $O(n + k)$ átlagosan, általánosan $O(n)$, legrosszabb esetben $O(n^2)$
 - k = edények száma
 - legrosszabb eset akkor van ha 1 edénybe kerül minden (nem egyenletes eloszlás)
 - egyenletes eloszlásnál hatékony (pl 0 és 1 közötti valós számok)

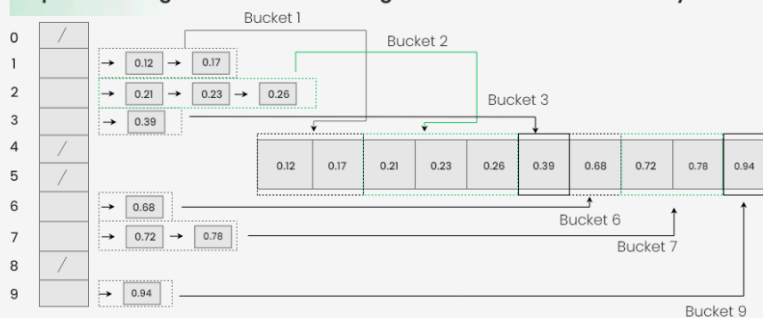
step 2 : Inserting Array elements into respective buckets



Step 3: Sorting individual Bucket



Step 4: Inserting buckets in ascending order into the resultant array



GRÁFALGORITMUSOK

Definíció

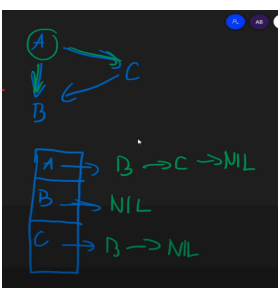
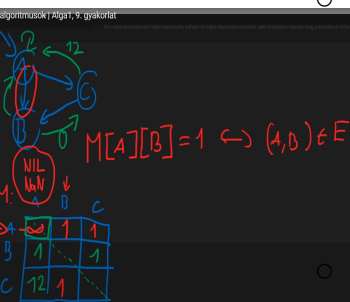
- a gráf $G = (V, E)$ csúcsok és élek halmaza

Típusai

- **Írányítatlan gráf:** az (u, v) él megegyezik a (v, u) éllel
- **Írányított gráf:** $u \rightarrow v$ él között van irányított út
 - erősen összefüggő
- **Súlyozott vagy Súlyozatlan:** élekhez számérték (súly) rendelődik-e vagy sem

Számítógépes reprezentáció

- **Egyszerű lista vagy tömb**
 - az összes (u, v) élt tartalmazza
 - tárigény: $\Theta(m)$
 - olyan algoritmusoknál használjuk, ahol az éleket sorba kell rendezni
- **Szomszédsági mátrix**
 - egy $V * V$ méretű mátrix, ahol az $A[i, j]$ elem értéke **1 (vagy a súly)**, ha van él egyébként 0 (vagy ha súlyozott akkor olyan kell amit nem vehet fel súlyként, pl null vagy ha csak pozitív súly lehet akkor akár -1)
 - pl $A \rightarrow B$, akkor $M[A][B] = 1$
 - ha irányítatlan a gráf akkor a szomszédsági mátrix főátlóra szimmetrikus lesz
 - előny: Gyors ($O(1)$) eldönteni, hogy két csúcs között van-e él
 - akkor jó választás sűrű gráfoknál, ahol m közel van n^2 -hez, vagy ha nagyon sokszor kell él létezésre rákérdezni
 - hátrány: Nagy a memóriaigénye ($O(V^2)$), akkor is ha kevés az él
- **Szomszédsági lista**
 - ha két csúcs között van él, akkor azokat felvesszük (irányítatlan)
 - ha az adott csúcsba vezet él, akkor azt felvesszük (irányított)
 - minden csúcshoz tartozik egy lista a szomszédairól
 - tömbbe annyi férőhely ahány csúcs van és minden retesz tárol egy láncolt listát
 - előny: Memóriahatékony ($O(V+E)$)
 - ritka gráfokhoz (keves él, $E \ll V^2$) ez a legjobb
 - gyakorlatban általában ez igaz
 - hátránya: Annak eldöntése, hogy létezik-e egy konkrét él két csúcs között lassabb $O(V)$, azaz $O(n)$



GRÁF KERESÉSEK (BEJÁRÁSOK)

Szélességi keresés (BFS)

- input: súlyozatlan gráf (akár irányított akár irányítatlan) és egy kezdőcsúcs
- output: egy szélességi fa
 - megtalálja a legrövidebb utat a kezdőcsúcsból, illetve az összes csúcsot melyet elérhetünk (és azt is, hogy legkevesebb hány éllel, szintekből látszik ez)
- sor (Queue) adaszerkezetet használ
 - FIFO - first in first out
- egy kezdőcsúcsból indulva rétegenként (mélységenként, sávosan) járja be a gráfot
- minden csúcsához meghatározásra kerül d (a távolság a kezdőcsúctól) és a szülője
- menete (A kezdőcsúccsal)
 - kezdőcsúcsot belerakjuk a sorba, kitöltjük a táblázatot az A -nál (kezdőelem) neki a kezdőelemtől vett távolsága 0 és nincs szülője (NIL) és szürkével jelöljük
 - kivesszük az A -t a sorból
 - megnézzük az A fehér szomszédait majd belerakjuk őket a sorba
 - fehér az amik még soha nem szerepeltek a sorban
 - kitöltjük őket a táblázatban (kezdőelemtől vett távolság és szülő csúcs) és szürkével jelöljük
 - szürke amik a sorba vannak
 - kivesszük a B -t a sorból, az A -t pedig feketével jelöljük (vele már nem foglalkozunk)
 - fekete amiket már feldolgoztunk
 - B fehér szomszédait is belerakjuk a sorba és táblázatot is kitöltjük majd jelöljük is őket szürkével
 - innen pedig így tovább, a fekete csúcsokat meg hagyjuk
- időigénye (mint a mélységi)
 - $O(V^2)$ - ha szomszédsági mátrixot használunk
 - $O(V + E)$ - ha szomszédsági listát használunk

1. FELADAT: SZÉLESSÉGI KERESÉS

• Hajtsuk végre a szélességi keresést a következő irányított gráfon az A kezdőcsúcsból

Q: $\{B, C, D\}$

A	0	NIL
B	1	A
C	1	A
D	2	B
E		
F		
G		
H		
I		
J		
K		

1. FELADAT: SZÉLESSÉGI KERESÉS

• Hajtsuk végre a szélességi keresést a következő irányított gráfon az A kezdőcsúcsból

Q: $\emptyset$

A	0	NIL
B	1	A
C	1	A
D	2	B
E	2	C
F	2	C
G	2	NIL
H	3	D
I	3	NIL
J	3	NIL
K	3	NIL

1. FELADAT: SZÉLESSÉGI KERESÉS

• Hajtsuk végre a szélességi keresést a következő irányított gráfon az A kezdőcsúcsból

Q: $\emptyset$

A	0	NIL
B	1	A
C	1	A
D	2	B
E	2	C
F	2	C
G	2	NIL
H	3	D
I	3	NIL
J	3	NIL
K	3	NIL

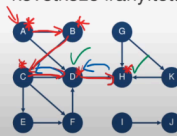
Mélységi keresés (DFS)

- input: súlyozatlan gráf (akár irányított akár irányítatlan) és egy kezdőcsúcs
- output: mélységi feszítőerdő
 - egy kezdőcsúcsból megtalálja a legrövidebb utat

- erdő, mert több fát is tartalmazhat akár
- feszítő, mert minden csúcsot érintenie kell (tartalmaznia kell)
- időigénye (mint a szélességének)
 - $O(V^2)$ - ha szomszédsági mátrixot használunk
 - $O(V + E)$ - ha szomszédsági listát használunk
- topologikus rendezésre és erősen összefüggő komponensek keresésére van
- verem (Stack) adatszerkezetet használ vagy legyakrabban rekurziót
 - LIFO - last in first out
- időbélyegek $[1, 2*V]$
 - **d[u] discovery time**: amikor a csúcsot felfedeztük (elérési idő)
 - **f[u] finish time**: amikor a csúcs vizsgálatát befejeztük (elhagyási idő)
- gráf **minden csúcsára** meghívásra kerül a mélységi bejárás
 - kezdetben minden csúcs fehér és szülője null, felveszünk egy T globális változót az elérési és elhagyási idők kezelésére
 - T globális változó növelése eggyel
 - az adott paramétercsúcs szürke lesz és a T az elérési ideje
 - az adott csúcs fehér szomszédjainak szülőjét az adott csúcsra állítsuk és indítsuk el rekurzivan a mélységi bejárást (minden lépésben a T eggyel nő)
 - mikor elértünk egy rész végéig elindulunk vissza és megnézzük egy csúcsra, hogy van-e még fehér szomszédja
 - ha nincs már fehér szomszédja, akkor az adott csúcsra az elhagyási időt T -re állítsuk és feketére fessük, ha van akkor mélységivel bejárjuk azt
 - fontos ha el is akadtunk nem állhatunk meg ha van még nem látogatott, ekkor ahová nem érünk el azt kiválasztjuk és az kerül bele

2. FELADAT: MÉLYSÉGI KERESÉS

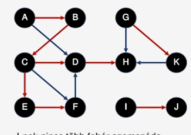
• Hajtsuk végre a mélységi keresést a következő irányított gráfon:



Handwritten notes: T, elhagyási idő

Csúcs	Szülő	Elérés	Elhagyás
A	NULL	1	14
B	A	2	13
C	B	3	12
D	C	4	7
E			
F			
G			
H	D	5	6
I			
J			
K			

2. FELADAT: MÉLYSÉGI KERESÉS



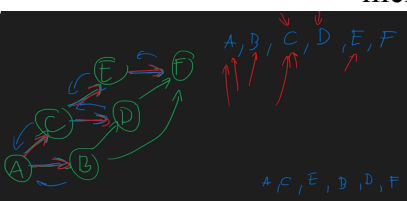
I-nak nincs több fehér szomszédja
T = 22
I-t elhagyjuk, mivel a gráfban nincs több fehér csúcs, végeztünk

Csúcs	Szülő	Elérés	Elhagyás
A	NULL	1	14
B	A	2	13
C	B	3	12
D	C	4	7
E	C	8	11
F	E	9	10
G	NULL	15	18
H	D	5	6
I	NULL	19	22
J	I	20	21
K	G	16	17

Handwritten lists:

- A → D → C → E → F
- G → K
- I → J

- **Topologikus rendezésre is alkalmas**
 - input: irányított, körmentes gráf
 - azért nem lehet kör benne mert ellentmondás lesz azzal kapcsolatban, hogy a van (u, v) él a gráfban akkor a sorozatban u előbb van mint v
 - pl $A \rightarrow B, C \rightarrow A, B \rightarrow C$ esetén értelemszerűen nem lehet mivel A úgy B előtt, hogy B-nek C előtt kéne és C-nek meg A előtt kéne lennie
 - menete
 - mélységi keresés során a csúcsok beszúrára kerülnek egy lista elejére elhagyásukkor (finish time)
 - fontos, hogy ha elakadunk kezdő csúcsból akkor folytatni kell a következő még nem vizsgált csúccsal
 - eredménye a csúcsok olyan sorozata (akár több is lehet egy gráfnak) mely a rendezett sorrendet tartalmazza és ha van (u, v) él a gráfban akkor a sorozatban u előbb van mint v

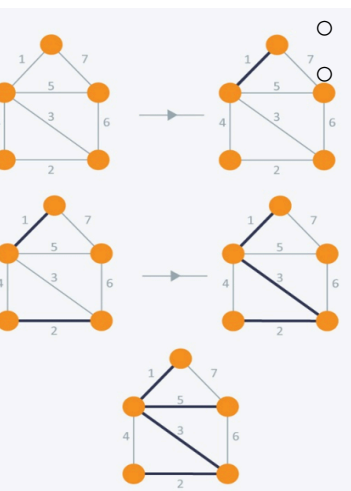


MINIMÁLIS FESZÍTŐFÁK (MST)

- input: összefüggő, irányítatlan, súlyozott gráf
- output: összefüggő, körmentes minimális feszítőfa (az élek összköltsége minimális)
 - feszítőfa = minden csúcsot érint
- erre két mohó algoritmus van: **kruskal algoritmus** és a **prim algoritmus**

Kruskal algoritmus

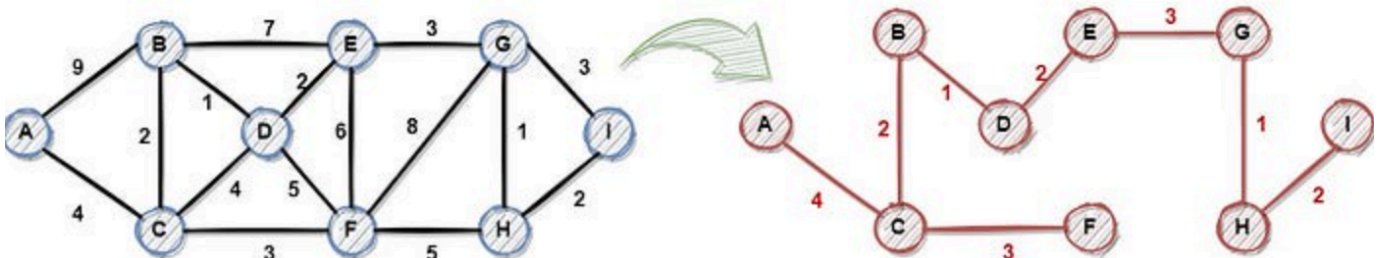
- több fát növeszt (minimális fák erdejét tárolja)
 - kezdetben minden csúcs egy fa
- mohó algoritmus
- időigénye: $O(E \log(E))$ - ez az élek rendezése
- menete



- élek súlyai szerint lesz egy rendezett listánk
- minden lépésben **azt a két különálló fát köti össze ami között a legkisebb a súly és ezeket egyesíti is**, ezzel minden lépéssel eggyel kevesebb fa lesz
 - fontos, hogy ha a legkisebb él két olyan csúcs között van ami azonos fában van azt kihagyjuk
 - segéd szerkezet: diszjunkt halmazok (kör detektálása, union-find adatszerkezet)

Prim algoritmus

- egy tetszőleges kezdőcsúcsból bővít EGY fát
 - akár több különböző helyes megoldása is lehet egyenlő élsúly esetén (a választott rendezési mód alapján pl: él nevek ábcé sorrendje, vagy szám esetén kisebbtől a nagyobbig stb)
- mohó algoritmus
- időigénye: $O(E \log(V))$ - bináris kupac esetén
 - sűrű gráfoknál előnyösebb ($E \gg V$, élek száma sokkal több mint a csúcsok száma, inkább már V^2 -hez közelít)
- menete
 - mindig **azt a csúcsot köti be a fába amelyik szomszédos a fa egyik csúcsával és a közöttük lévő él a legkisebb**, ezzel minden lépéssel eggyel kevesebb elérhető csúcs van
 - nem kezdőcsúctól való távolság van csak a szomszédos csúcsok közöttit nézzük
 - segédszerkezete: a prioritási sor

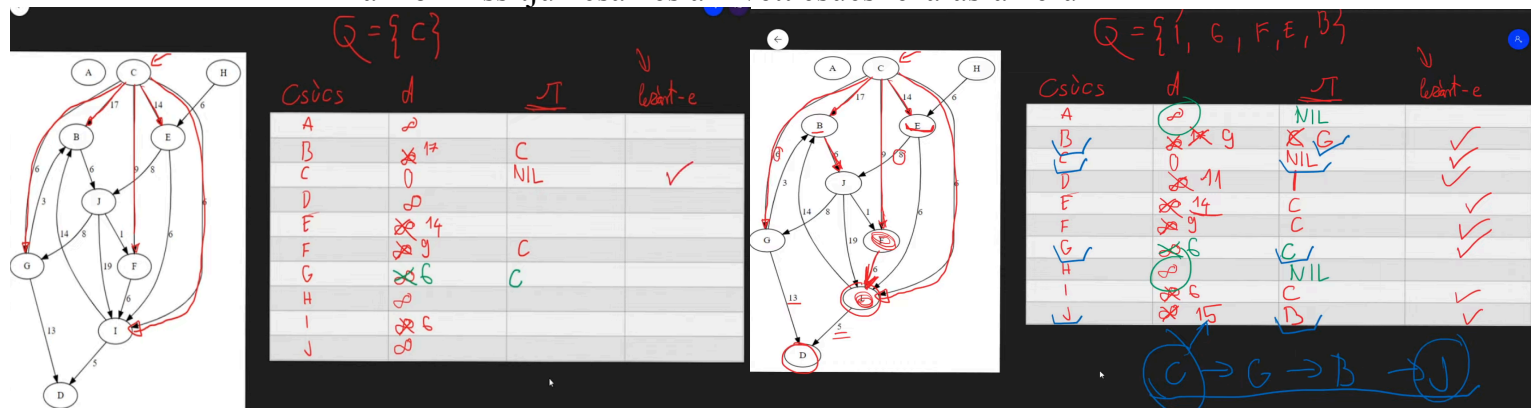


LEGRÖVIDEBB UTAK GRÁFOKNÁL

- több algoritmus típus is van rá:
 - adott csúsból induló legrövidebb utak: **Dijkstra** és **Bellman-Ford**
 - legrövidebb utak minden csúcspárra: **Ford-Walshall**

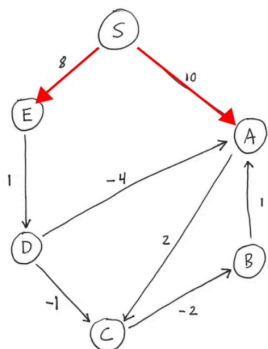
Dijkstra algoritmus

- mohó algoritmus
- input: összefüggő, súlyozott gráf, negatív összköltségű kör nélkül, általánosan negatív költségű él nélkül vesszük
 - a negatív súlyú élek nem okoznak problémát, de a negatív összköltségű kör esetén viszont nem megoldható (mert a körhöz jutva végtelen ciklusba esik)
 - összköltség = az összes él súlyának összege, pl. ha 1 2 -2 -1 esetén 0, ez még jó de -1 -2 -2 -1 már nem mert -6 ami által negatív összköltségű lesz
- output: adott kezdőcsúsból határozzuk meg a legrövidebb utat minden csúcshoz
- időigénye egy csúcsra: $O(E * \log V)$
 - minden csúcsra: $O(V * (E * \log V))$
- menete, hasonlít a szélességi keresésre
 - felvesszünk egy táblát, ami tárolja a távolságot (d) a kezdőcsúctól, és a csúcs őse (pi) és, hogy lezárt-e
 - alaphoz a távolság végtelen (kezdőcsúcs kivétel, ott 0), az őse pedig null
 - veszünk egy Q prioritási sort és először belerakjuk a kezdőcsúcsot
 - majd mindig kivesszük ebből a prioritási sorból a legkisebb a távolságú csúcsot mely még nem lezárt, ez alapján frissítésre kerül ennek az összes szomszédjának a távolsága (relaxáció) és a szülője, ha a szomszéd értéke már meghatározott volt (de még nem lezárt) akkor ha javulna (kisebb lennek) akkor frissítjük csak és a kivett csúcs lezárásra kerül



Bellman-Ford algoritmus

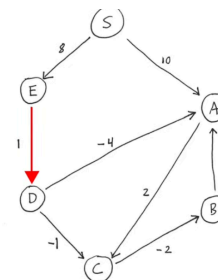
- input: egy kezdő csúcs és egy irányított, súlyozott gráf, de itt már negatív összköltségű körökön is működik (return -1) és ezeket detektálni is tudja (V-1 edik iterációban is még javítana akkor negatív kör van)
- output: adott kezdőcsúcsból határozzuk meg a legrövidebb utat minden csúcshoz
- időigénye egy csúcsra: $O(V * E)$
 - lassabb mint a Dijkstra
 - minden csúcsra: $O(V^2 * E)$
- tárigénye: $O(V)$
- mohó algoritmus
- legrosszabb esetben $V - 1$ iteráció van
 - felvesszünk egy táblát, ami tárolja a távolságot (d), és a csúcs őse (pi)
 - a kezdő csúcs távolsága 0, míg a többi végtelen lesz
 - egy iterációban végig megyünk minden csúcson és próbálunk minden él esetén javítani
 - ha a csúcshoz tartozó távolság végtelen, azaz nem fedeztünk még fel utat oda akkor kihagyjuk
 - ha van az adott csúcshoz út akkor a szomszédos csúcsainál megnézzük, hogy javul-e így a távolság, ha javul akkor frissítjük ennek a távolságát és a szülő csúcsát
 - amennyiben egy iterációban nem változik a táblázat akkor megkaptuk a legrövidebb értékeket



	10		8		
0	∞	∞	∞	∞	∞
S	A	B	C	D	E

1st Iteration

eközött minden csúcson is átment



				9	
0	10	10	12	∞	8
S	A	B	C	D	E

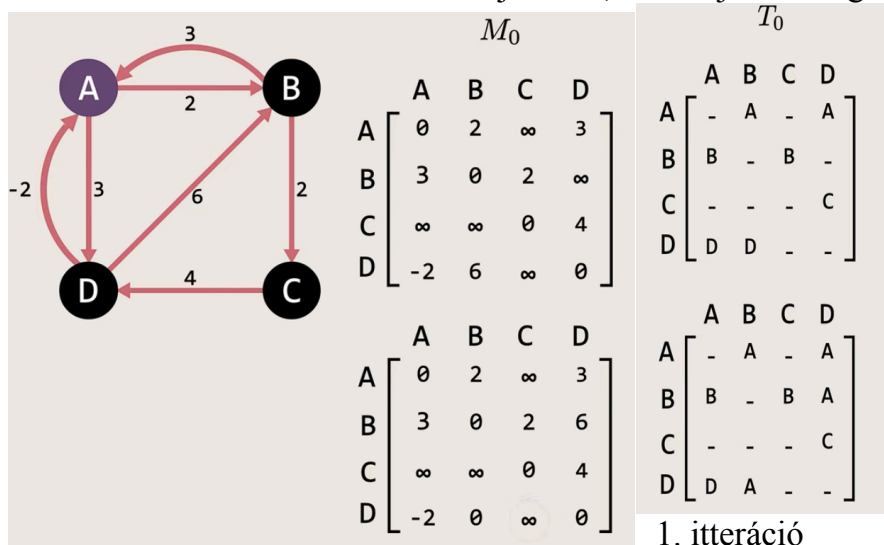
1st Iteration

Legrövidebb utak keresése körmentes gráfban (DAG)

- input: körmentes gráf
- menete
 - gráfot topológikusan rendezzük (lásd fentebb)
 - csúcsok sorrendjét kapjuk, ahol minden él előre mutat
 - topológikus sorrendet követve minden csúcsból induló élet relaxáljuk

Floyd-Warshall algoritmus

- input: irányított vagy irányítatlan súlyozott gráf
- output: legrövidebb utakat adja meg minden csúcspárra
- akkor választjuk:
 - ha vannak negatív élsúlyok, de nincsenek negatív élsúlyú körök
 - ha sűrű a gráf
- időigénye: $O(V^3)$
 - három egymásba ágyazott ciklus / csúcs
- tárigénye: $O(V^2)$ - a távolságmátrix tárolása
- alapja a dinamikus programozás (részfeladatokra bontás)
- d mátrix, ahol d_{ij} eleme az ismert legrövidebb út i -ből j -be
- működése
 - egy $d(i,j)^k$ értéket számol ki, ami i -ből j -be vezető legrövidebb út hossza, ahol az út közbülső csúcsai csak az $\{1, 2, \dots, k\}$ halmazból kerülhetnek ki
 - k -t 1-től V -ig növeljük, és minden lépésben megnézzük az i -ből j -be vezető út rövidebb-e, ha átmegyünk k -adik csúcson, vagy ha nem
 - i az a B csúcs, j az a D és a $k=1$ akkor megnézem, hogy $B \rightarrow A \rightarrow D$ -vel tudok-e javítani, ha az új útköltség így kisebb akkor frissítjük



FUTÁSIIDŐK

	Legjobb	Átlagos	Legrosszabb	Tár	Stabil
Beszűrő	n	n^2	n^2	1	✓
Összefésülő	$n \log n$	$n \log n$	$n \log n$	n	✓
Kupac	$n \log n$	$n \log n$	$n \log n$	1	✗
Bináris Fa	$n \log n$	$n \log n$	$n \log n$	1	✓
Gyors	$n \log n$	$n \log n$	n^2	1	✗
Leszámoló	$n+k$	$n+k$	$n+k$	nk	✓
Számjegyes	$d(n+k)$	$d(n+k)$	$d(n+k)$	nk	✓
Edény	n		n^2	n	✓